



MULTI-VECTOR CYBER DECEPTION PLATFORM

Honeyman V2

A distributed, mobile honeypot fleet that turns a Raspberry Pi into a portable tripwire for physical, wireless, and network attacks.

Architecture & V1 → V2 overview

· Research / non-commercial

PLATFORM OVERVIEW

What Honeyman is

Honeyman turns a Raspberry Pi into a portable, multi-sensor tripwire you can drop into any hostile environment — a conference floor, a hotel, a client site — and watch for attacks across **five vectors from one global dashboard.**



Mobile sensors

Runs on a Raspberry Pi you can carry into the field. Ports 1–2 stay normal; ports 3–4 are armed as honeypot triggers.



One-command onboarding

A single curl installs a sensor and self-registers it to the dashboard with auto-generated credentials. No accounts to create.



Five attack surfaces

USB, Bluetooth LE, Wi-Fi, AirDrop / AWDL, and multi-protocol network honeypots — all in a single unit.



Global visibility

Every sensor streams to a central collector; a live world map shows fleet status and threats as they land.

DETECTION COVERAGE

Five attack surfaces, one sensor



USB

BadUSB • O.MG cables
Rubber Ducky •
payloads

Flags HID-injection
cables posing as
ordinary USB.



Bluetooth LE

Flipper Zero • spoofing
beacon flooding •
cloning

Reads advertisements &
RSSI, not just device
names.



Wi-Fi

Evil-twin • deauth
floods
rogue APs • karma

SSID / BSSID correlation
with graceful tool
fallback.



AirDrop / AWDL

mDNS / Bonjour recon
AWDL • sharingd
crashes

Behavioral, protocol-
level signals over name
matching.

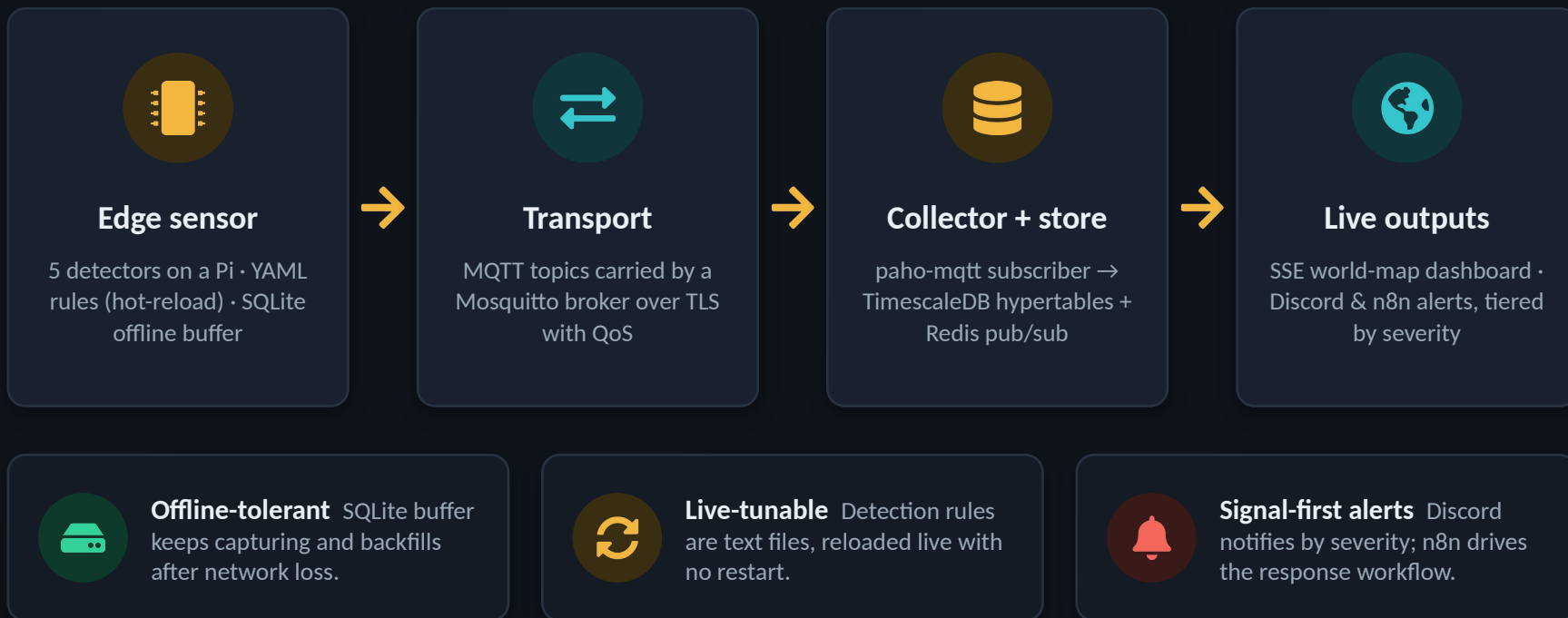


Network

SSH • Telnet • SMB
MySQL • Redis • HTTP/S

OpenCanary honeypots
catch scans & credential
theft.

How an attack becomes an alert



From one box to a fleet



V1 • Honey1

Single-device proof of concept

- Single Raspberry Pi, monolithic controller
- Local ELK / Kibana dashboard (localhost)
- Threat logs stay on the device
- Keyword / signature detection
- Email, syslog & webhook alerts
- One operator, one location



V2 • Honeyman

Distributed sensor platform

- Fleet of mobile Pi sensors
- One curl to deploy & self-register
- MQTT → central collector
- TimescaleDB + live world map
- Behavioral + protocol detection
- Offline-tolerant, multi-site

SIDE BY SIDE

V1 vs V2, dimension by dimension

Dimension	V1 • Honey1	V2 • Honeyman
Topology	Single device	Distributed sensor fleet
Onboarding	Manual setup script	One curl • self-registers • auto creds
Transport	Local files / syslog	MQTT over Mosquitto (TLS)
Storage	Elasticsearch (ELK)	TimescaleDB hypertables + Redis
Dashboard	Local Kibana	Global SSE world map
Detection	Keyword signatures	Behavioral + protocol-level
Alerting	Email / syslog / webhook	Discord by severity + n8n
Resilience	Online only	SQLite buffer, offline-tolerant
Scale	One node	Tens to hundreds (phased)

Decisions we made along the way



MQTT over HPFeeds

Mature paho-mqtt client with TLS & QoS and huge headroom on a tiny footprint; HPFeeds scales only vertically with a smaller ecosystem.



TimescaleDB over InfluxDB

PostgreSQL / SQL familiarity, hypertables and continuous aggregates, and ~90% compression for long-term threat retention.



Bridge net + alt ports

Host networking collides with real services; bridge mode with SSH→2222 and HTTP→8080 keeps the honeypots conflict-free.



YAML rules, hot-reload

Detection logic lives as text files that can be edited and reloaded live, without restarting a sensor in the field.



SQLite edge buffer

Sensors keep capturing through network loss and backfill to the core once connectivity returns.



Discord + n8n alerting

Rich, severity-tiered notifications and workflow-driven incident response, instead of plain email.

From signatures to behavior

Attacker tooling is built to slip past name and keyword matching, so V2 leans on **how a device behaves** and **how a protocol misbehaves**.



The stealth USB implant

An O.MG cable looks like a plain USB 2.0 cable but hides a Wi-Fi-capable chip that injects keystrokes as an HID — dormant until triggered. V2 flags HID + storage combos and unexpected input devices rather than trusting appearances.



USBGuard-style allowlisting

Following USBGuard's model, the armed ports treat every new device as suspect and run deep analysis — authorize the known, quarantine the unknown, right at the honeypot edge.



Protocol cadence

BLE advertises every couple of seconds and is prone to fuzzing / DoS crashes. V2 parses advertisement data and watches cadences — like the ~0.5 Hz sharingd crash-retry behind known AirDrop CVEs.

ROADMAP

On the horizon



In build now

- BLE Continuity / Nearby advertisement parser
- Crash-cadence network detector for sharingd-style retry loops



Next

- AirDrop decoy receiver in "Everyone" mode with plausible names
- Malformed plist-bomb detection
- Quick Share / Nearby Share handshake anomalies



Later

- Geo dashboard: country → city-level precision
- Multi-Continuity service correlation
- USBGuard-style sender allowlist

SUMMARY

The short version

- ✓ V1 proved the detection modules on one Pi; V2 turns them into a deployable fleet.
- ✓ One curl to deploy — MQTT and TimescaleDB carry and keep the data; a live world map shows it.
- ✓ Detection shifted from fragile keyword signatures to behavioral, protocol-level indicators.
- ✓ Built to keep working offline, alert by severity, and grow from tens to hundreds of sensors.